SHL Conversational Assessment Recommender

Internship Project Summary | Aditya

1. Project Overview

For this assignment I had to build a conversational API that recommends SHL assessments to a hiring manager, based on what they type about the role they are hiring for. The idea was to combine an LLM with a retrieval system (RAG) so the model doesn't just make up assessment names, it actually pulls them from the real SHL catalog. The API needed to be able to:

- understand hiring requirements from the conversation
- ask clarification questions when the request is vague
- recommend relevant SHL assessments
- update recommendations if requirements change mid-conversation
- compare assessments when asked
- expose all of this through a FastAPI REST API

2. Initial Approach

My first version used semantic search with SentenceTransformer for embeddings and FAISS as the vector database. I went with this because embeddings usually give better retrieval than plain keyword matching – they can match meaning even when the wording is different (e.g. "coding test" vs "programming assessment"). This part worked fine locally.

3. Problems I Faced

Deployment memory issue

Render's free tier only gives 512 MB RAM. SentenceTransformer downloads a fairly large embedding model, and FAISS added more memory on top of that. Every time I deployed, it failed with out-of-memory errors, "no open ports detected", or the service crashing before it even finished starting. After several failed deployments I realised the problem wasn't a bug in my code – the architecture itself was too heavy for a free-tier server.

Pandas metadata error

While installing dependencies on Render I also hit a metadata generation error related to pandas. I fixed this by updating the pandas version in requirements.txt and rebuilding the environment from scratch.

JSON catalog issues

When I started testing retrieval, I noticed the SHL catalog JSON itself had problems – missing fields, inconsistent URLs, duplicate entries, and formatting that wasn't uniform. I had to go through it and clean/standardize the JSON before retrieval results became reliable.

Local development issues

Along the way I also ran into the usual development pain: missing packages, dependency conflicts, build failures, wrong imports, retriever methods that broke after I refactored the code, and random API errors while testing. Nothing major on its own, just a lot of small fixes over several days of debugging.

Gemini API issues

Gemini worked fine during development, but later I started getting 429 RESOURCE_EXHAUSTED / quota exceeded errors because of the free-tier API limits. I added error handling so the API fails gracefully with a proper message instead of crashing when this happens.

4. Why I Changed the Retrieval System

FAISS gave better semantic retrieval, but it was clear deployment reliability mattered more – a system that recommends well but can't stay online is useless. So I switched to TF-IDF with Cosine Similarity instead. It uses much less memory, starts up fast, runs fine on Render's free tier, and still gave relevant recommendations in my testing. It's a simpler method, but it made the whole API stable and actually deployable, which was the real goal.

5. Final System Architecture

User Request → Conversation Analyzer → Conversation State Extraction → Query Builder → TF-IDF Retrieval → Cosine Similarity Ranking → Relevant SHL Assessments → Google Gemini → Final API Response

6. Prompt Design

The prompt tells Gemini to only recommend assessments that came back from the retriever – it's not allowed to invent assessment names or URLs. It also has to explain why each assessment was recommended, only compare assessments that were actually retrieved, and ask a clarification question if the requirement given is too vague to search on.

7. Evaluation

I tested the API manually with different hiring scenarios – Java Developer, Data Analyst, personality-based hiring, leadership roles, comparing two assessments, changing requirements mid-conversation, and general multi-turn chats. For each one I checked retrieval quality, whether recommendations actually made sense, whether Gemini stayed grounded to the catalog, the raw API responses, and how the conversation flow held up over multiple turns.

8. Final Deployment

The app is deployed on Render. It exposes GET /health for a health check, POST /chat for the main conversation endpoint, and Swagger docs at GET /docs. One thing to note: since it's on Render's free tier, the service sleeps when idle, so the first request after inactivity has a cold-start delay of a few seconds.

9. Future Improvements

- Hybrid retrieval (BM25 + embeddings) once memory isn't a constraint
- A proper ranking model instead of plain cosine similarity
- A lightweight vector database that fits free-tier hosting
- Caching for repeated queries
- Persistent conversation memory across sessions
- More structured evaluation metrics instead of manual testing
- Tighter prompt engineering for edge cases

10. Reflection

This project taught me a lot more about the practical side of shipping something than about the AI part itself. I learned about conversational AI and retrieval systems, but honestly most of my time went into debugging, dependency management, and deployment constraints I hadn't thought about before starting. Switching away from my original FAISS setup felt like a step back at first, but the final version is more reliable and actually usable, which matters more than having the "better" architecture on paper.